

Guía de la API REST

Sistema de Movilidad

Aventura Vip de Cusco

Backend: Node.js + Express + TypeScript

Base de datos: PostgreSQL 15

Versión de la API: 1.0.0

Módulo implementado: Autenticación (Login)

Documento generado automáticamente

Guía de la API REST

Sistema de Movilidad

Aventura Vip de Cusco

Backend: Node.js + Express + TypeScript

Base de datos: PostgreSQL 15

Versión de la API: 1.0.0

Módulo implementado: Autenticación (Login)

Tabla de contenidos

1	Introducción
2	Arquitectura del sistema
3	Requisitos previos
4	Instalación y puesta en marcha
4.1	Base de datos (PostgreSQL en Docker)
4.2	Servidor API
5	Estructura del proyecto
5.1	Registro de peticiones con Morgan
6	Base de datos
6.1	Tablas del esquema
6.2	Procedimientos almacenados
6.3	Cuentas de prueba
7	Autenticación con tokens JWT
8	Endpoints de la API
8.1	POST /api/auth/login
8.2	GET /api/auth/me
9	Ejemplos de uso con curl
10	Códigos de estado y errores
11	Buenas prácticas de seguridad
12	Aplicación web (frontend) y modo PWA
12.1	Frontend (React + Vite)
12.2	Probar en el celular (red local)
12.3	PWA instalable con túnel HTTPS (cloudflared)
13	Próximos pasos

1. Introducción

Este documento es la **guía de uso de la API REST** del sistema de movilidad de la empresa **Aventura Vip de Cusco**, la cual brinda servicio de taxis corporativo al grupo Belmond (hoteles Monastery y Nazarenas).

Esta primera versión implementa el módulo de **Autenticación (Login)**: la parte del sistema que valida las credenciales de un usuario y emite un **token JWT** que identifica a la persona en todas las peticiones futuras.

El backend está desarrollado en **Node.js con Express y TypeScript**, y usa **PostgreSQL 15** como base de datos. La lógica de negocio vive en procedimientos almacenados de PostgreSQL, por lo que la API es una "capa fina" que conecta las peticiones HTTP con la base de datos.

2. Arquitectura del sistema

El flujo completo del login funciona de la siguiente manera:

```

Cliente (curl / futura app)
|
| 1. POST /api/auth/login {correo_o_dni, clave}
v
Servidor Node.js (Express) --- src/modules/auth/auth.controller.ts
| (morgan registra cada peticion HTTP en la consola)
|
| 2. Llama al procedimiento SELECT SP_LOGIN($1, $2)
v
PostgreSQL 15 --- valida credenciales, devuelve datos + permisos (JSONB)
|
| 3. Respuesta JSON
v
Servidor: firma el token JWT (12 horas de vigencia)
|
| 4. Respuesta HTTP 200 {token, usuario}

```

```

v
Cliente: guarda el token y lo envia en el encabezado Authorization
|
| 5. GET /api/auth/me (Authorization: Bearer <token>)
v
Middleware autenticarToken valida la firma y expiracion del token
|
v
Se devuelven los datos del usuario autenticado

```

3. Requisitos previos

Para ejecutar este proyecto se necesita:

- **Node.js** versión 18 o superior (incluye npm).
- **Docker** (para correr PostgreSQL 15 sin instalarlo en el sistema).
- **Python 3** (opcional, solo para regenerar este PDF).
- **curl** o **Postman** para probar los endpoints.

4. Instalación y puesta en marcha

4.1. Base de datos (PostgreSQL 15 en Docker)

Levantar el contenedor con PostgreSQL 15:

```

docker run --name av-test-pg -e POSTGRES_USER=aventura \
-e POSTGRES_PASSWORD=aventura123 -e POSTGRES_DB=aventura_vip \
-p 5432:5432 -d postgres:15-alpine

```

Crear e inyectar los scripts SQL (en este orden obligatorio):

```

cd "back-end"
npm run db:init

```

O ejecutar manualmente cada script en orden:

```

docker exec -i av-test-pg psql -U aventura -d aventura_vip < db/00_SCHEMA.SQL
docker exec -i av-test-pg psql -U aventura -d aventura_vip <
db/02_AUDIT_TRIGGER.SQL
docker exec -i av-test-pg psql -U aventura -d aventura_vip <
db/01_PROCEDURES.SQL
docker exec -i av-test-pg psql -U aventura -d aventura_vip < db/03_SEEDS.SQL

```

El orden importa porque cada script usa lo que crea el anterior:

- 00_SCHEMA.SQL - crea las tablas.
- 02_AUDIT_TRIGGER.SQL - crea los triggers de auditoría.
- 01_PROCEDURES.SQL - crea los procedimientos almacenados.
- 03_SEEDS.SQL - inserta roles, empresas y cuentas de prueba.

4.2. Servidor API

Instalar las dependencias y compilar el TypeScript:

```

cd "back-end"
npm install
npm run build

```

Levantar el servidor (dos opciones):

```
npm start      # ejecuta la versión compilada (recomendado en producción)
npm run dev    # modo desarrollo (se recarga al guardar)
```

El servidor escucha en **http://localhost:3000**.

Configuración (archivo .env):

```
PORT=3000
DATABASE_URL=postgres://aventura:aventura123@localhost:5432/aventura_vip
JWT_SECRET=aventura_vip_super_secreto_2026_cambiar_en_produccion
JWT_EXPIRES_IN=12h
```

5. Estructura del proyecto

```
aventura-vip/
+-- AGENTS.md                      # Guías de trabajo
+-- back-end/
|   +-- .env / .env.example        # Variables de entorno (no publicar)
|   +-- package.json              # Dependencias y scripts (db:init completo)
|   +-- tsconfig.json             # Configuración de TypeScript
|   +-- db/
|       +-- 00_SCHEMA.SQL          # Esquema: 7 tablas e índices
|       +-- 01_PROCEDURES.SQL      # SP_LOGIN, SP_REGISTRAR_USUARIO, etc.
|       +-- 02_AUDIT_TRIGGER.SQL   # Triggers de auditoría (MODIFICADO_AT)
|       +-- 03_SEEDS.SQL           # Datos iniciales (9 permisos, cuentas demo)
|       +-- 05_USUARIOS_CRUD.SQL   # CRUD de usuarios (SPs)
|       +-- 06_ORGANIZACION.SQL    # Empresas, establecimientos, áreas (SPs)
|       +-- 07_SP_EXTRA.SQL        # SP_OBTENER_CATALOGOS, SP_OBTENER_ROL
|   +-- tests/                    # 35 pruebas de integración (vitest)
|   +-- docs/                     # Esta guía y el informe de tesis
|   +-- src/
|       +-- server.ts              # Punto de entrada (arranca el servidor)
|       +-- app.ts                 # Aplicación Express (helmet, CORS, rutas)
|       +-- config/
|           +-- database.ts        # Pool de conexiones a PostgreSQL
|           +-- env.ts             # Validación de variables de entorno
|       +-- middlewares/
|           +-- auth.middleware.ts  # Validación de tokens JWT
|           +-- autorizarPermiso.ts # Permisos PBAC (403 si falta)
|           +-- error.middleware.ts # Manejo de errores (AppError)
|       +-- controllers/
|           +-- auth.controller.ts  # Login y /me
|           +-- usuarios.controller.ts # CRUD de usuarios
|           +-- catalogos.controller.ts # Catalogo único (SP)
|           +-- organizacion.controller.ts # Empresas/establecimientos/áreas
|       +-- routes/
|           +-- auth.routes.ts / usuarios.routes.ts / catalogos.routes.ts /
|           +-- organizacion.routes.ts
|       +-- modules/
|           +-- auth/ - usuarios/  # Tipos/contratos por módulo
+-- front-end/
+-- index.html                     # Punto de entrada HTML
+-- vite.config.ts                 # Vite + PWA + proxy /api + darkMode
+-- tailwind.config.js             # Paleta oficial (negro, marrón, ambar)
+-- public/
|   +-- Logo.svg                   # Logo vectorial de la marca
|   +-- fonts/                     # Azonix y Caviar Dreams (WOFF2)
|   +-- icono-*.png                # Iconos de la PWA
+-- src/
+-- main.tsx                       # Arranque de React (contextos)
+-- App.tsx                        # Componente raíz
+-- config/                        # api.ts, navegacion.tsx, presets.ts
```

```

+-- context/          # Tema (oscuro/claro), favoritos, navegacion
+-- hooks/            # usePermisos, useMediaQuery
+-- layouts/          # DashboardLayout, Sidebar, Header
+-- services/         # auth, usuarios, organizacion
+-- router/index.tsx  # Rutas protegidas (token + permisos)
+-- pages/            # Login, Inicio, Usuarios, UsuarioForm, Organizacion,
Perfil
+-- components/       # Loader, LogoVip, CampoFormulario, ui/
+-- types/index.ts    # Tipos de la API

```

5.1. Registro de peticiones con Morgan

El backend utiliza **morgan**, un middleware de Express que **registra en la consola cada petición HTTP** que recibe la API. Esto facilita el monitoreo, la depuración y el diagnóstico de errores en tiempo real.

Al ejecutar el servidor, cada petición genera una línea de log con el método HTTP, la ruta, el código de estado y el tiempo de respuesta:

```

GET / 200 3.247 ms - 154
POST /api/auth/login 200 101.699 ms - 751
POST /api/auth/login 401 24.025 ms - 36

```

El color de la salida también es útil: **verde** indica éxito (2xx) y **amarillo** indica advertencias (4xx, como un login rechazado).

6. Base de datos

6.1. Tablas del esquema

Tabla	Descripción
ROLES	Roles del sistema: ADMIN, OPERADOR, CONDUCTOR, USUARIO.
EMPRESAS	Organizaciones: Aventura Vip de Cusco (proveedor), Belmond (cliente), Particular (app pública).
ESTABLECIMIENTOS	Sedes de una empresa (hoteles de Belmond: Monasterio, Palacio Nazarenas, Rio Sagrado).
AREAS	Sub-áreas de un establecimiento (Recepción, A&B, Cocina, Housekeeping).
USUARIOS	Todas las personas: empleados, conductores y clientes.

Tabla	Descripción
PERMISOS	Catálogo de permisos del sistema (PBAC, 9 permisos).
USUARIOS_PERMISOS	Asignación de permisos a usuarios (N:M).

Todas las tablas principales incluyen campos de auditoría: CREADO_AT, CREADO_POR, MODIFICADO_AT y MODIFICADO_POR. Las fechas se llenan automáticamente (el trigger actualiza MODIFICADO_AT en cada edición).

6.2. Procedimientos almacenados

El 100% de las consultas de la API se ejecutan mediante procedimientos almacenados:

Procedimiento	Función
SP_REGISTRAR_USUARIO	Crea un usuario (cifra la clave con bcrypt).
SP_LOGIN	Valida credenciales según el tipo de cuenta (DNI/correo) y devuelve datos y permisos.
SP_VALIDAR_USUARIO	Verifica que un usuario siga activo.
SP_LISTAR_USUARIOS SP_OBTENER_USUARIO	/ Listado con búsqueda/filtros y detalle con permisos.
SP_ACTUALIZAR_USUARIO SP_CAMBIAR_ESTADO_USUARIO	/ Edita datos (clave opcional) y activa/inactiva usuarios.
SP_ASIGNAR_PERMISO SP_QUITAR_PERMISO SP_REEMPLAZAR_PERMISOS	/ / Administra los permisos de un usuario.
SP_GUARDAR_EMPRESA SP_GUARDAR_ESTABLECIMIENTO SP_GUARDAR_AREA	/ / Crean o editan los catálogos de la organización.

Procedimiento	Función
SP_CAMBIAR_ESTADO_* / SP_LISTAR_* (organización)	Estado y listados de empresas, establecimientos y áreas.
SP_OBTENER_CATALOGOS / SP_OBTENER_ROL_POR_NOMBRE	Catálogos en un JSONB y rol por nombre.
SP_ACTUALIZAR_MODIFICADO_AT	Trigger: actualiza la fecha de modificación.

6.3. Cuentas de prueba

Todas las cuentas usan la clave demo123 y los trabajadores inician sesión con su **DNI**.

DNI (acceso)	Correo	Rol	Empresa	Reserva
12345678	admin@aventura.com	ADMIN	Aventura Vip de Cusco	No
23456789	operador@aventura.com	OPERADOR	Aventura Vip de Cusco	No
34567890	jorge@aventura.com	CONDUCTOR	Aventura Vip de Cusco	No
45678901	juan.perez@belmond.com	USUARIO	Belmond (Hotel Monasterio)	Sí
56789012	maria.tupac@belmond.com	USUARIO	Belmond (Palacio Nazarenas)	No

7. Autenticación con tokens JWT

El sistema usa **JSON Web Tokens (JWT)** para la autenticación:

- Cuando un usuario inicia sesión con credenciales válidas, el servidor firma un token con la clave secreta

JWT_SECRET.

- El token dura **12 horas** (configurable en JWT_EXPIRES_IN).
- El token contiene el id del usuario, su rol y sus permisos. Así la API no necesita consultar la base en cada petición.
- Para acceder a rutas protegidas, el cliente envía el token en el encabezado: Authorization: Bearer <token>.
- El middleware autenticarToken verifica la firma y la expiración; si algo falla, responde HTTP 401.

Regla de identificador de acceso (validada en SP_LOGIN):

- Trabajadores de Aventura Vip y de los hoteles (Belmond) - **DNI** (8 dígitos).
- App pública (empresa "Particular") - **correo electrónico**.
- El login tiene **rate limit**: 5 intentos por IP por minuto (HTTP 429).

8. Endpoints de la API

8.1. POST /api/auth/login

Propósito: inicia sesión y devuelve el token de acceso.

Autenticación requerida: ninguna (endpoint público).

Cuerpo de la petición (JSON):

Campo	Tipo	Obligatorio	Descripción
correo_o_dni	string	Sí	Correo electrónico o DNI del usuario.
clave	string	Sí	Contraseña en texto plano.

Respuesta exitosa (HTTP 200):

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...\"",
  "expira_en": "2026-08-11T11:51:06.000Z",
  "usuario": {
    "id": 1,
    "nombres": "Admin",
    "apellidos": "Sistema",
    "correo": "admin@aventura.com",
    "dni": "12345678",
    "telefono": "999999999",
    "rol": "ADMIN",
    "empresa": "Belmond",
    "area": null,
    "empresa_id": 1,
    "area_id": null,
    "puede_reservar": false,
    "permisos": ["CREAR_RESERVAS", "DESPACHAR_COLA", "..."]
  }
}
```

Respuestas de error:

Código	Mensaje	Cuándo ocurre
400	Faltan datos...	No se envió correo_o_dni o clave.
401	Credenciales invalidas	Usuario no existe, inactivo o clave incorrecta.
500	Error interno del servidor	Fallo inesperado.

8.2. GET /api/auth/me

Propósito: devuelve los datos del usuario autenticado.

Autenticación requerida: sí (token JWT en el encabezado).

Cuerpo de la petición: ninguno.

Encabezado requerido: Authorization: Bearer <token>

Respuesta exitosa (HTTP 200):

```
{
  "id": 1,
  "rol": "ADMIN",
  "permisos": ["CREAR_RESERVAS", "DESPACHAR_COLA", "..."]
}
```

Respuestas de error:

Código	Mensaje	Cuándo ocurre
401	No se envió un token...	Falta el encabezado Authorization.
401	Token inválido o expirado	Token falso, manipulado o vencido.
401	Usuario inactivo o no existe	El usuario fue desactivado tras emitir el token.

8.3. Gestión de usuarios - /api/usuarios

Propósito: CRUD del módulo de usuarios. Permisos requeridos:

Endpoint	Método	Permiso requerido
/api/usuarios	GET	GESTIONAR_USUARIOS o CREAR_USUARIOS
/api/usuarios/:id	GET	GESTIONAR_USUARIOS o CREAR_USUARIOS
/api/usuarios	POST	GESTIONAR_USUARIOS o CREAR_USUARIOS (con solo CREAR_USUARIOS solo rol USUARIO)
/api/usuarios/:id	PUT	GESTIONAR_USUARIOS o CREAR_USUARIOS (con solo CREAR_USUARIOS solo clientes)
/api/usuarios/:id/estado	PATCH	GESTIONAR_USUARIOS
/api/usuarios/:id/permisos	POST / DELETE	GESTIONAR_USUARIOS

8.4. Catálogos - /api/catalogos

Propósito: devuelve roles, empresas, establecimientos, áreas y permisos para alimentar los formularios (una sola llamada a SP_OBTENER_CATALOGOS). Requiere sesión (cualquier usuario autenticado).

8.5. Organización - /api/organizacion

Propósito: gestión de empresas, establecimientos y áreas. Requiere el permiso **GESTIONAR_CATALOGOS** (solo admin).

Endpoint	Método	Función
/api/organizacion/empresas	GET / POST	Listar / crear empresas

Endpoint	Método	Función
/api/organizacion/empresas/:id	PUT / PATCH estado	Editar / activar o inactivar
/api/organizacion/establecimientos	GET / POST	Listar / crear establecimientos
/api/organizacion/establecimientos/:id	PUT / PATCH estado	Editar / activar o inactivar
/api/organizacion/areas	GET / POST	Listar / crear áreas
/api/organizacion/areas/:id	PUT / PATCH estado	Editar / activar o inactivar

9. Ejemplos de uso con curl

9.1. Iniciar sesión (trabajador, por DNI)

```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"correo_o_dni":"12345678","clave":"demo123"}'
```

9.2. Iniciar sesión (app pública, por correo)

```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"correo_o_dni":"usuario@app.com","clave":"demo123"}'
```

9.3. Consultar el perfil (/me) con el token

Paso 1: guardar el token en una variable:

```
TOKEN=$(curl -s -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"correo_o_dni":"12345678","clave":"demo123"}' \
| python3 -c "import sys,json;print(json.load(sys.stdin)['token'])")
```

Paso 2: usarlo en el endpoint protegido:

```
curl http://localhost:3000/api/auth/me \
-H "Authorization: Bearer $TOKEN"
```

9.4. Ejemplo de error: credenciales incorrectas

```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"correo_o_dni":"12345678","clave":"clave_equivocada"}'
# Respuesta: {"mensaje":"Credenciales invalidas"} (HTTP 401)
```

9.5. Ejemplo de error: sin token

```
curl http://localhost:3000/api/auth/me
# Respuesta: {"mensaje":"No se envió un token de acceso"} (HTTP 401)
```

9.6. Ejemplo de error: rate limit (fuerza bruta)

```
# Tras 5 intentos fallidos en un minuto, la API responde:
# {"mensaje":"Demasiados intentos de acceso. Espera un momento..."} (HTTP 429)
```

9.7. Listar usuarios (requiere permiso GESTIONAR_USUARIOS o CREAR_USUARIOS)

```
curl "http://localhost:3000/api/usuarios?q=&rol=&estado=" \
-H "Authorization: Bearer $TOKEN"
```

9.8. Prueba rápida del servidor

```
curl http://localhost:3000/
# Respuesta: {"nombre":"API Aventura Vip de Cusco","version":"1.0.0","estado":"Servidor en línea",...}
```

10. Códigos de estado y errores

Código	Significado	Uso en esta API
200	OK	Petición exitosa (login, /me, listados).
201	Creado	Recurso creado (usuario, empresa, establecimiento, área).
400	Solicitud incorrecta	Faltan campos obligatorios o formato inválido.

Código	Significado	Uso en esta API
401	No autorizado	Credenciales inválidas o token ausente/vencido/revocado.
403	Prohibido	El usuario no tiene el permiso requerido (PBAC).
404	No encontrado	La ruta o el recurso solicitado no existe.
409	Conflicto	Correo/DNI duplicado o nombre repetido en el mismo nivel.
429	Demasiadas peticiones	Se superó el rate limit del login (5 intentos/min).
500	Error interno	Fallo inesperado del servidor.

11. Buenas prácticas de seguridad

- Las claves se almacenan **cifradas con bcrypt** (nunca en texto plano).
- El servidor **nunca devuelve la clave** en las respuestas.
- Los errores de login usan el mismo mensaje genérico ("Credenciales invalidas") para no revelar si el identificador o la clave fue lo incorrecto.
- helmet** añade cabeceras HTTP seguras (CSP, X-Frame-Options, HSTS).
- CORS restringido** a los orígenes configurados en CORS_ORIGENES (en producción se agrega el dominio del sistema).
- Rate limit** en el login (5 intentos por IP por minuto, HTTP 429).
- Control de acceso **por permisos (PBAC)**: cada endpoint declara el permiso que exige y el middleware responde HTTP 403 si falta.
- El archivo .env con JWT_SECRET **no debe subirse** a repositorios públicos; en producción usar un gestor de secretos.
- Los tokens expiran (12 h); los usuarios inactivos pierden el acceso aunque su token siga "vigente" (validación con SP_VALIDAR_USUARIO).
- El 100% de las consultas se ejecutan mediante **procedimientos almacenados** con parámetros (protección contra inyección SQL).
- En producción se debe servir la API por **HTTPS**.

12. Aplicación web (frontend) y modo PWA

12.1. Frontend (React + Vite)

El frontend es una SPA (Single Page Application) desarrollada en **React** con **Vite**, **Tailwind CSS** y **TypeScript**. Usa **axios** para llamar a la API, **react-router-dom** para la navegación y **vite-plugin-pwa** para convertir la app en una **PWA (Progressive Web App)**: instalable y funcional sin conexión.

Instalar las dependencias y arrancar el servidor de desarrollo:

```
cd "front-end"
npm install
npm run dev
```

El servidor de desarrollo escucha en **http://localhost:5173** y redirige todas las peticiones `/api` hacia el backend (puerto 3000) mediante `server.proxy` en `vite.config.ts`.

Generar el build de producción (incluye manifest y service worker):

```
npm run build
npm run preview
```

El build genera `dist/sw.js` (service worker), el `dist/manifest.webmanifest` y el registro automático (`dist/registerSW.js`).

12.2. Probar en el celular (red local)

Opción A: abrir la app con el navegador de un celular en la misma red Wi-Fi.

1. En `vite.config.ts` debe estar `server.host: true` para exponer el servidor hacia la red local (ya configurado).
2. Conocer la IP local del Mac: `ipconfig getifaddr en0`.
3. En el celular abrir `http://IP_DEL_MAC:5173`.

El login y toda la app funcionan normalmente, pero **no se puede instalar**: los navegadores exigen HTTPS para habilitar el service worker y la instalación.

12.3. PWA instalable con túnel HTTPS (cloudflared)

Opción B: exponer la app por HTTPS con un túnel gratuito de Cloudflare (`cloudflared`) para que el navegador permita **instalar la app** y usarla sin internet.

1) Generar el build de producción y servirlo:

```
cd "front-end"
npm run build
npm run preview -- --host
```

2) Descargar el binario de `cloudflared` (no requiere Homebrew; se obtiene de GitHub Releases y se coloca en `~/local/bin`): en Apple Intel usar `cloudflared-darwin-amd64.tgz`; en Apple Silicon usar `cloudflared-darwin-arm64.tgz`.

```
curl -L -o cloudflared.tgz
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-darwin-amd64.tgz
tar -xzf cloudflared.tgz
chmod +x cloudflared
mv cloudflared ~/local/bin/
```

3) Abrir el túnel hacia el preview (puerto 4173):

```
~/local/bin/cloudflared tunnel --url http://localhost:4173
```

4) Del log del túnel tomar la URL pública `https://XXXX.trycloudflare.com` y abrirla en el celular. Ahí ya aparece la instalación: en Android/Chrome desde el menú "Instalar aplicación"; en iPhone/Safari en "Compartir" y luego "Añadir a pantalla de inicio".

Si el preview responde **HTTP 403** para el host del túnel, es porque Vite bloquea nombres de host desconocidos; se resuelve con `preview.allowedHosts: true` en `vite.config.ts`.

La URL del túnel es **temporal** (cambia al reiniciar el túnel) y, mientras esté activa, la app queda expuesta a Internet; conviene cerrar el túnel al terminar de probar (`kill cloudflared`).

13. Próximos pasos

El módulo de autenticación queda listo. Las siguientes fases serán:

- **Gestión de usuarios:** CRUD con `SP_REGISTRAR_USUARIO` y asignación de permisos.
- **Reservas:** creación y control de viajes (vales de Belmond).
- **Flota y conductores:** vehículos, disponibilidad y validación del código secreto.
- **Cola nocturna:** orden de despacho y GPS.